



Visão geral de PLN

Você vai aprender sobre os conceitos e elementos fundamentais relacionados ao processamento de linguagem natural (PLN). Essa é uma das áreas da inteligência artificial (IA) que tem crescido bastante e, claramente, tem muitas aplicações práticas no mercado.

Prof. Sérgio Monteiro

Propósito

É importante usar o Google Colab para desenvolver os exemplos na linguagem Python. Ele é gratuito. Tudo que você vai precisar é de uma conta no Gmail e, claro, praticar os exemplos e exercícios que propusermos. Os códigos dos exercícios desenvolvidos estão disponíveis neste [link](#).

Objetivos

- Reconhecer a construção da linguagem natural: tokenização, marcação de parte do discurso e reconhecimento de entidade nomeada.
- Identificar técnicas de PLN: CBOW e Skip-Gram.
- Implementar práticas de PLN em Python.

Introdução

A IA tem feito parte das nossas vidas com maior frequência. Por exemplo, ao ligarmos para uma operadora de telefone, somos recepcionados por um sistema de IA e, a partir daí, somos guiados sobre qual o caminho mais eficiente que devemos seguir para prosseguir nosso atendimento.

Uma das áreas da IA mais proeminente nesse tipo de aplicação é o processamento de linguagem natural (PLN) – inclusive, será o foco do nosso estudo. Dessa forma, estaremos nos preparando para atuar em uma das áreas que, muito em breve, vai demandar profissionais altamente qualificados.

PLN: visão geral

Assista ao vídeo e entenda como o processamento da linguagem natural (PLN) se destaca como uma das áreas da inteligência artificial (IA) e algumas de suas aplicações práticas.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

PLN: uma das áreas da IA

O PLN é um campo da IA que tem como foco a extração de dados e, muitas vezes, de informações e de textos através do uso de algoritmos específicos. Ele tem uma grande importância prática, pois é bastante comum que as pessoas produzam conteúdo relevante sem seguir um padrão específico. Assim, é bastante difícil que os métodos tradicionais possam obter essas informações e, assim, gerar valor significativo.

Não é à toa que com a popularidade da internet, o PLN ganhou mais importância, pois, como sabemos, as pessoas geram muito conteúdo todos os dias em sites, blogs, fóruns, redes sociais etc. Muitas dessas fontes podem ser úteis para diversas aplicações e o PLN nos permite – através de seus algoritmos e técnicas – processar e analisar grandes quantidades de dados textuais.

A IA, por sua vez, é muito mais ampla que o PLN e cobre diversas áreas, tanto que PLN é apenas uma das diversas aplicações possíveis. A seguir, vamos ver algumas das aplicações de PLN que, certamente, vão nos ajudar a entender como utilizá-la para resolver problemas reais.

Extração e recuperação de informações

Uma das aplicações de PLN é a extração de informações de dados textuais não estruturados. Esses textos podem ser, dentre outras fontes:

- Documentos
- Artigos
- Feeds de mídia social

Neles, podemos identificar entidades, relacionamentos e sentimentos importantes. Podemos utilizar essas informações para minerar dados, realizar mecanismos de pesquisa, fazer sistemas de recomendação e analisar qual a percepção dos usuários sobre determinado produto ou serviço, o que é chamado na literatura de **análise de sentimentos**.



Exemplo

Podemos encontrar em um texto nomes de pessoas, de equipamentos e componentes, como também obtermos datas relevantes em que determinados eventos ocorreram.

Tradução automática

Certamente, um dos maiores avanços da humanidade foi a comunicação. Na prática, temos diversos idiomas e, apesar de termos dicionários à disposição para nos auxiliar no processo de tradução de idiomas, sabemos que é bem mais complexo do que isso, pois cada língua tem uma estrutura muito peculiar.

As técnicas de PLN podem nos auxiliar através de sistemas de tradução automática. Dessa forma, é bem mais simples podermos entender o significado de um texto dentro de um contexto e não apenas o significado de uma palavra. Isso é útil de várias formas, por exemplo, facilitar a comunicação global e aprimorar a colaboração intercultural em vários domínios, como: negócios, educação e diplomacia.

Sumarização de texto e geração de linguagem

Há muitas situações em que precisamos das informações mais importantes de documentos muito extensos. Nesse caso, o PLN nos ajuda a extrair o essencial dessas informações. Obviamente, de modo algum isso invalida a necessidade de estudarmos o documento com mais atenção posteriormente, mas é bastante útil para recuperação rápida de informações, de forma que os usuários tenham uma rápida compreensão das principais informações.

Outro ponto muito interessante em que o PLN pode ser aplicada é na geração de linguagem natural. Inclusive, esse tipo de aplicação está muito popularizado em se tratando de:

- Geração automática de relatórios
- Criação de conteúdo
- Mensagens personalizadas
- Respostas de chatbot

Certamente, isso só é possível porque são muitas as situações que se repetem, por serem passíveis de automatização, vide o atendimento automático a fim de se obter informações de cartão de crédito.

Compreensão da linguagem e compreensão contextual

Em casos mais sofisticados, o PLN nos ajuda a compreender a linguagem humana, o que envolve análise sintática e semântica. Atualmente, temos diversos buscadores na internet e ferramentas de chat que interagem com as pessoas como se fossem seres humanos. Ou seja, esses sistemas utilizam técnicas avançadas que permitem distinguir as ambiguidades que podem existir em um texto.

Uma outra aplicação nessa mesma área é em relação à saúde. Por exemplo, um paciente pode relatar diversos sintomas e, a partir da análise deles em conjunto, um sistema de PLN pode auxiliar um profissional de saúde a realizar uma análise mais precisa sobre os exames e o tipo de tratamento que devem ser aplicados.

De forma semelhante, também podemos utilizar técnicas de PLN para segurança pública, em que os boletins de ocorrência são registrados em texto-livre. Assim, podemos extrair dados e informações que nos permitam gerar estatísticas e traçar perfis de situações que, de outra forma, seria muito difícil de realizar.

Atividade 1

É um fato que o PLN pode ser aplicado em diversas áreas. No entanto, é fundamental entendermos que é necessário que existam estudos e, obviamente, investimentos específicos para obter uma qualidade satisfatória. Nesse sentido, selecione a opção que contém uma justificativa para o fato de o PLN apresentar grandes desafios sobre a tradução de textos para outros idiomas.

A

O PLN ainda é muito recente e tem um escopo bastante limitado, por enquanto.

B

A IA usa o PLN apenas para textos estruturados.

C

O PLN não pode ser aplicada em textos em que haja ambiguidade.

D

Os idiomas possuem detalhes que precisam de um contexto para serem compreendidos.

E

A tradução de idiomas é algo muito simples e, por isso, não desperta interesses do desenvolvimento de algoritmos de PLN.



A alternativa D está correta.

Na tradução de textos, não é suficiente apenas traduzir as palavras, pois elas podem ter muitos significados. Além disso, existem peculiaridades regionais que tornam o significado de uma frase bastante difícil de ser traduzido para outra linguagem. Isso faz com que as técnicas de PLN precisem ficar cada vez mais sofisticadas, para realizar boas traduções.

Tokenização

Assista ao vídeo a aprenda sobre a importância da técnica de tokenização, além de conferir alguns exemplos práticos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição e aplicações de tokenização

Tokenização é o processo de dividir um texto em unidades menores chamadas tokens. Esses tokens podem ser palavras individuais, frases ou até mesmo caracteres. No PLN, a tokenização faz parte da etapa de pré-processamento e tem vários propósitos importantes. Entre esses processos estão:

1

Segmentação de texto

Consiste em dividirmos um fluxo contínuo de texto em unidades, como palavras ou frases. Isso é essencial, pois muitos algoritmos e modelos de PLN utilizam estrutura do texto para termos uma compreensão sobre o significado de palavras dentro das frases.

2

Criação de vocabulário

É fundamental uma vez que, através do PLN, podemos criar vocabulários ou dicionários de tokens relacionados ao contexto que estamos trabalhando. Importante entendermos algumas pontos: um token representa um conceito específico e ter um vocabulário padronizado permite um processamento de dados, análise e treinamento de modelo mais fáceis.

3 Normalização de texto

O objetivo é converter as palavras do texto para um formato mais simples de ser processado. Por exemplo, podemos remover acentos de palavras, ou convertê-las em minúsculas. Outro exemplo comum é remover flexões e contrações, para que as palavras fiquem mais simples de serem padronizadas e, assim, termos uma quantidade relevante de conteúdo para gerar estudos e análises relevantes.

4

Extração de características

Os tokens podem ser usados para encontrarmos sequências vizinhas de n tokens, por exemplo, a frase “copo com água” tem significado complementar à frase “copo com doce”. Além disso, podemos extrair tags de parte da fala.

5

Eficiência computacional

Pode melhorar a eficiência computacional, pois os algoritmos passam a trabalhar com tokens em vez de textos inteiros. Dessa forma, podemos usar estruturas de dados que sejam vantajosas na forma como os dados são organizados no texto e, assim, obter respostas mais rápidas e precisas.

De um modo geral, a tokenização permite transformarmos dados de texto não estruturados em um formato estruturado que pode ser efetivamente processado, analisado e usado como entrada para vários modelos de aprendizado de máquina e aprendizado profundo.

Alguns exemplos de tokenização

Agora, que já sabemos o que é tokenização, vejamos alguns exemplos práticos!

1

Tokenização de palavras

Entrada: “Eu estudo processamento de linguagem natural.”

Saída: ['Eu', 'estudo', 'processamento', 'de', 'linguagem', 'natural', '.']

2

Tokenização de sentença

Entrada: “Eu gosto de correr. A pista de corrida está ótima hoje.”

Saída: ["Eu gosto de correr.", "A pista de corrida está ótima hoje."]

3

Tokenização de caracteres

Entrada: “Olá, mundo!”

Saída: ['O', 'l', 'á', ',', ' ', 'm', 'u', 'n', 'd', 'o', '!']

4 Reconhecimento de entidade nomeada

Entrada: 'Carlos Chagas está entre os maiores cientistas do Brasil.'

Saída: [(“Carlos Chagas”, “Pessoa”), (“Cientista”, “Título”), (“Brasil”, “País”)]

Esses são exemplos de tokenização, que é praticamente a porta de entrada para o mundo de PLN. Então é importante entender esses conceitos iniciais antes de avançar. Quando tiver segurança, faça o exercício para consolidar seus conhecimentos. Bons estudos e vamos em frente!

Atividade 2

A ideia de tokenizar um texto é relativamente simples. No entanto, quando avançamos um pouco no estudo, percebemos que não é apenas uma questão de separar as palavras, mas de organizá-las de tal modo que sejam úteis para outras fases das técnicas de PLN. Nesse sentido, selecione a opção correta que contém uma aplicação em que a tokenização é muito útil.

A

Gerar dados para análise estatística.

B

Facilitar a estruturação dos dados com listas.

C

Eliminar palavras repetidas.

D

Utilizar técnicas de IA para verificar se um texto é fake news.

E

Verificar a autoria de um texto.



A alternativa A está correta.

A tokenização é uma tarefa extremamente útil para o PLN. Ele ajuda a transformar textos não estruturados em um formato que seja possível organizar, de modo a encontrarmos relações interessantes, caso existam, e se possível, encontrar estatísticas relevantes que permitam que outras etapas das técnicas de PLN possam utilizar mais adiante.

Marcação de parte do discurso

Assista ao vídeo e aprenda do que trata a marcação de parte do discurso, além de conferir alguns exemplos práticos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição de marcação de parte do discurso

A marcação de parte da fala, do inglês *part of speech* (POS), é um processo no PLN que envolve a atribuição de tags ou rótulos gramaticais a cada palavra em um texto. O objetivo é indicar a que categoria sintática o texto, ou parte dele, se refere. Através desses rótulos, podemos obter informações sobre o papel e a função de cada palavra em uma frase.

A marcação POS é importante no PLN, pois nos ajuda em dois aspectos:

1

Entender a estrutura sintática de uma frase.

2

Entender a estrutura semântica de uma frase.

A sintaxe está relacionada com a estrutura da frase. Por exemplo, na língua portuguesa, uma oração é composta por sujeito, verbo e predicado. Um sujeito pode assumir várias categorias: simples, composto, oculto, indeterminado e sem sujeito. Isso é sintaxe!

Por outro lado, a semântica está relacionada ao significado dentro da frase. Por exemplo, “o carro é predominante vermelho”. Mas o que isso significa? É algo bom ou ruim? Predominante significa quantos por cento em relação às outras cores do carro? Ou seja, é uma análise muito mais elaborada, mas que depende da análise sintática para que possa ser feita.

Alguns exemplos de marcação de parte do discurso

Agora, vamos apresentar alguns exemplos de POS na língua portuguesa!

Substantivo

Representa pessoa, lugar, coisa ou ideia. Exemplos: casa, gato, computador etc.

Adjetivo

Modifica um substantivo, caracterizando-o. Exemplos: bonita, pequena, feliz etc.
check

Pronome

Pode ser usado no lugar de um substantivo para evitar repetições. Exemplos: eu, você, aquele, isso etc.

Verbo

Expressa uma ação, ocorrência ou um estado. Exemplos: correr, programar, estudar, ler etc.

Preposição

Relaciona um substantivo (ou pronome) e outras palavras em uma frase, indicando local, tempo ou modo. Exemplos: entre, contra, desde, em etc.

Conjunção

Conecta palavras ou frases. Exemplos: e, ou, mas, nem, porém, todavia etc.

Advérbio

Modifica um verbo, adjetivo ou outro advérbio, fornecendo mais informações sobre modo, tempo, lugar, grau etc. Exemplos: rapidamente, agora, pouquíssimo etc.

Interjeição

Expressa forte emoção ou surpresa, podendo ser representado por uma palavra ou frase. Exemplos: Ah!, Oh!, Ei!, Eh! Etc.

Esses são apenas alguns exemplos das classes gramaticais em português. Cada idioma tem suas próprias POS e, ainda, existem as variações relacionadas a regionalismos, que são bem mais complexas de serem capturadas por técnicas de PLN. O essencial por enquanto é entendermos que o PLN, especificamente POS, preocupa-se com a estrutura (sintaxe) e o significado (semântica) do texto para compreender o que está sendo expresso e como podemos agir a partir de uma informação relevante.

Atividade 3

Provavelmente, você já deve ter assistido a filmes em que as pessoas (normalmente, vítimas) ligam para uma central de emergência. Geralmente, a pessoa que faz o atendimento tenta registrar o máximo possível de informações para poder ajudar. Agora, trazendo essa situação para o mundo real, selecione a opção correta como a técnica de POS pode ser útil no atendimento a uma vítima.

A

Corrigindo erros de digitação que são bem comuns em situações em que as pessoas estão sob forte pressão.

B

Eliminando a possibilidade de trotes ao detectar contradições no pedido de socorro da suposta vítima.

C

Oferecendo suporte para identificar pessoa, local, objetos e outras informações úteis para obter maior precisão sobre o cenário, e auxiliar melhor a vítima e a pessoa do atendimento.

D

A POS pode determinar a prioridade da situação e automaticamente acionar uma equipe para oferecer suporte para a vítima.

E

Eliminar a necessidade de ter atendentes humanos, pois são caros e, normalmente, sujeitos a cometer erros que um algoritmo não faria.



A alternativa C está correta.

É totalmente absurdo supor que um sistema pode substituir um profissional de atendimento em um setor de emergência. Existem diversos fatores que, simplesmente, não podem ser programados e que só podem ser realizados por profissionais competentes, treinados e devidamente remunerados. Para a situação descrita, a POS seria muito útil para obter informações de tal forma que os agentes de apoio poderiam agir com maior precisão.

Reconhecimento de entidade nomeada

Assista ao vídeo e aprenda como reconhecer entidades nomeadas e como isso pode ser útil em diversas situações práticas.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição de entidades nomeadas

Uma entidade pode ser qualquer coisa que possamos identificar. Por exemplo, uma entidade pode ser uma pessoa, um veículo, um imóvel, entre tantas outras coisas que possamos classificar. Portanto, deve ficar muito claro para nós, pois é um assunto extremamente importante dentro das aplicações de PLN. Ao reconhecermos uma entidade dentro de um texto, podemos dar um tratamento adequado. Esse tratamento pode se traduzir

em estudo estatístico, descoberta de relações não triviais entre entidades e padrões de comportamentos, além de ser útil em diversas áreas, como segurança, saúde e meio ambiente.

Quando aplicamos uma análise de PLN para identificar entidades nomeadas de textos, podemos ter diversos objetivos, mas, certamente, o mais importante deles é o de extrair informações significativas do texto que nos ajudem a entender as relações entre essas entidades e, quando possível, associar esse entendimento a comportamentos, locais e periodicidades. Como ferramenta, ela fornece capacidades que podem ser usadas tanto de forma ética como antiética. Por isso, sempre devemos respeitar a legislação e agir de forma responsável acerca do que nossas ações podem produzir.

Alguns exemplos de entidades nomeadas

É bem simples encontrar exemplos de entidades nomeadas. A seguir, apresentamos alguns exemplos para ficar bem concreto o nosso entendimento.

Pessoa

Carlos Chagas, César Lattes, Jacob Palis etc.

Organização

Petrobras, Supremo Tribunal Federal, Organização das Nações Unidas etc.

Localização

Fortaleza, Rio de Janeiro, São Paulo etc.

Data

26 de outubro, 26 de março de 2022 etc.

Horário

10h00, 16h30 etc.

Dinheiro

R\$ 100,00 (cem reais), \$ 1,5 milhão de dólares etc.

Porcentagem

20%, 75,5% etc.

Produto

Livro, computador, carro etc.

Evento

Jogo de futebol, campeonato de karatê etc.

Idioma

Inglês, espanhol, francês etc.

Condição médica

Diabetes, asma, estresse, fadiga, depressão etc.

Título do livro

Bíblia, Elementos de Topologia Geral etc.

Nesse momento, o nosso objetivo é apenas entender o que é o reconhecimento de uma entidade nomeada e perceber, através dos exemplos, como ele pode ser extremamente importante. Mas existem diversas técnicas que podemos aplicar para fazer esse reconhecimento. Realmente, é essencial que tenhamos em mente que uma entrada típica para esse tipo de aplicação é um texto-livre e a saída apresentará uma ou mais entidades com sua(s) respectiva(s) classificação(ões). Por exemplo, observe o caso a seguir.

- **Entrada:** “Noam Chomsky é um pesquisador extremamente relevante tanto na linguística como na computação.”
- **Saída:** ([“Noam Chomsky”], [“PESSOA”])

Nesse exemplo, o sistema de reconhecimento de entidade nomeada identificou “Noam Chomsky” como uma pessoa.

Não é à toa que o reconhecimento de entidades nomeadas é tão importante no estudo de PLN. E não podemos esquecer: estamos trabalhando, na maioria dos casos, com textos não estruturados. Isso adiciona ainda mais um desafio nesse trabalho, no entanto, a recompensa vale a pena, pois esse conhecimento nos permite uma melhor compreensão, extração e análise de dados de texto e, claro, isso agrega valor para os dados que, por sua vez, podem ser muito úteis para o suporte à tomada de decisão.

Atividade 4

Muitos profissionais, ainda hoje em dia, realizam trabalho de campo em que precisam fazer anotações em papéis. Isso parece ser um pouco estranho para quem convive com tecnologia o tempo todo, mas há muitos locais em que há limitações de energia e é inviável usar equipamentos como notebooks ou outros eletrônicos. Analisando esse caso, selecione a opção que contém a explicação correta sobre o uso de reconhecimento de entidade nomeada.

A

Faz sentido, pois os dados serão digitados posteriormente, de forma não estruturada em um sistema, e podem revelar relações importantes entre entidades.

B

Faz sentido, pois é natural que o texto seja colocado posteriormente em um sistema de dados estruturados que vai aumentar a confiança sobre as informações obtidas.

C

Faz sentido, pois, atualmente, é bastante comum que as próprias pessoas que coletam dados sejam capazes de fazer classificações deles. Isso torna o trabalho de identificação de entidades bem simples.

D

Não faz sentido, pois é muito difícil colocar dados de origem de anotações em sistemas computacionais.

E

Não faz sentido, pois, se a empresa realmente precisasse das informações sobre as entidades, ela usaria um sistema eletrônico, mesmo com limitações.



A alternativa A está correta.

O Brasil é um país gigantesco e com muitas peculiaridades. Existem muitas atividades que ainda são registradas em papéis por vários motivos que vão da falta de investimento em tecnologia até limitações técnicas. Então, quando esses dados são registrados em forma de texto-livre em um sistema computacional é o momento adequado de aplicar técnicas de PLN como o reconhecimento de entidades nomeadas para tentar extrair valor e, assim, oferecer mais informações que forneçam suporte para a tomada de decisão, se for possível.

A revolução do PLN

Assista ao vídeo e aprenda sobre as diversas áreas que as técnicas de PLN revolucionaram e o que vem pela frente.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Pelo que já estudamos até agora, podemos ter clareza de que as técnicas de PLN têm muitas aplicações práticas. Obviamente, isso trouxe uma grande revolução na comunicação entre humanos e computadores. Acompanhe alguns desses avanços!

1

Interação humano-computador

Atualmente, dispomos de assistentes de voz, chatbots e assistentes virtuais sofisticados que são capazes de compreender melhor o que o humano deseja e produzir uma resposta mais clara, aprimorando, assim, a experiência e a acessibilidade do usuário.

2

Tradução de idiomas e comunicação multilíngue

Cada vez mais, temos técnicas que permitem, além da tradução de idiomas, a própria tradução da fala de um idioma para outro. Ainda há muito a ser feito nessa área, mas, certamente, isso vai promover muitas oportunidades de colaboração entre povos de línguas distintas.

3

Recuperação e extração de informações

Vemos esse tipo de aplicação nos buscadores da WEB constantemente. O usuário realiza uma consulta e o sistema fornece resultados de pesquisa mais relevantes.

4Análise de sentimento

O objetivo é determinar o sentimento ou a emoção que os usuários expressam em texto a respeito de determinado assunto. Na WEB, existem muitos locais para coletar esses dados e eles são usados em aplicações de monitoramento de mídia social, gerenciamento de reputação de marca, pesquisa de mercado e análise de opinião pública.

5

Geração de texto e criação de conteúdo

Aplicações mais avançadas de PLN permitem analisar de forma automática um conjunto de fontes de informações e fazer geração de texto, ou seja, fazer a criação, de forma automatizada, de conteúdo. É óbvio que essa área ainda é muito nova e demanda muita atenção para evitar que se espalhem notícias falsas.

Tendências de PLN para os próximos anos

Bem, falamos sobre o que já existe sobre PLN, mas podemos esperar um pouco mais daqui um tempo?

Certamente, com o avanço dos algoritmos e da tecnologia e, principalmente, da demanda de novos serviços, há muito espaço para as aplicações de PLN contribuírem. Na sequência, vamos conferir algumas das tendências que visualizamos para muito em breve.

1

Melhor compreensão da linguagem

Os modelos de PLN continuarão a evoluir e permitirão uma melhor comunicação entre pessoas e sistemas.

2

Sistemas personalizados e conscientes do contexto

Os modelos se tornarão mais personalizados e, assim, terão um comportamento mais contextualizado ao meio com o qual ele está interagindo.

3

Interatividade com outras tecnologias e técnicas

As técnicas de PLN possivelmente interagirão com imagens, vídeos e dados de sensores.

4

Avanços nos aspectos éticos

Certamente, haverá muitos debates e avanços na legislação sobre o uso das técnicas de PLN. Devemos nos lembrar de que estamos falando de um poderoso instrumento que pode ser usado com excelentes objetivos, ou de forma extremamente antiética e, até mesmo, criminosa.

5

Domínio específico

Aplicações de PLN para saúde, setor jurídico, área financeira e, de um modo geral, para pesquisa científica, vão continuar a avançar.

Existem muitas outras aplicações para além dessas que mencionamos. Essa é uma área muito interessante e que pode ser muito útil para a sociedade, desde que venha acompanhada de treinamentos, investimentos em saúde, educação e segurança e, claro, com avanços significativos na legislação, para evitar “ditadores da tecnologia”.

Atividade 5

Não há dúvidas de que as técnicas de PLN já fazem parte da nossa vida e que ainda vão avançar bastante. No entanto, quando falamos sobre tendências de avanço dessa área, fomos muito insistentes na questão da necessidade do debate ético e dos avanços da legislação. Nesse sentido, selecione a opção que contém uma forma antiética de uso das técnicas de PLN.

A

Usar um atendente virtual de PLN para dar suporte a uma conta de energia elétrica.

B

Aplicar PLN para auxiliar um profissional da saúde.

C

Disponibilizar uma aplicação de PLN que auxilie os alunos a estudar PLN.

D

Receber comandos e acionar dispositivos pré-programados.

E

Espionar as conversas das pessoas sem autorização judicial.



A alternativa E está correta.

As técnicas de PLN precisam ser úteis para a sociedade. Então, é esperado que auxiliem profissionais nas diversas áreas e ajudem estudantes a acelerar a curva de aprendizado. Por outro lado, é inaceitável utilizar esse tipo de técnica para obter informações sobre as pessoas sem que haja consentimento explícito ou autorização judicial.

As diversas técnicas para PLN

Assista ao vídeo e aprenda sobre algumas das principais categorias de técnicas de PLN, além de entender como elas podem ser combinadas para produzir resultados escaláveis e úteis.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Categorias de técnicas para PLN

Nós podemos utilizar diversas técnicas para o processamento de linguagem natural (PLN) com os objetivos de realizar processamentos eficientes, encontrar relações não triviais, fazer correções de texto, gerar a linguagem humana entre outras diversas aplicações. Na sequência, vamos destacar um pouco melhor algumas das principais técnicas aplicadas em PLN.

Tokenização

Consiste na divisão do texto em unidades menores.

Normalização do texto

Transforma o texto em um formato padronizado. Basicamente, temos duas técnicas aqui:

- Lematização: Consiste em reduzir palavras à forma base. Por exemplo, as palavras “aprendendo” ou “aprendido” tem como forma lematizada a palavra “aprender”.
 - Stemização: Reduz palavras à forma raiz. Por exemplo, a palavra “aprendendo” se transforma em “aprend”.
-
- Lematização: Consiste em reduzir palavras à forma base. Por exemplo, as palavras “aprendendo” ou “aprendido” tem como forma lematizada a palavra “aprender”.
 - Stemização: Reduz palavras à forma raiz. Por exemplo, a palavra “aprendendo” se transforma em “aprend”.

Marcação de parte do discurso (POS)

Consiste em tags gramaticais ou rótulos das palavras em um texto para identificar sua categoria sintática, por exemplo, em um substantivo, verbo ou adjetivo.

Reconhecimento de entidade nomeada

Identifica e classifica as entidades nomeadas no texto, como pessoas, organizações, locais, datas etc.

Análise de sentimento

Determina o sentimento ou a emoção predominante em um trecho de texto. Por exemplo, positivo, negativo e neutro.

Classificação de texto

Categoriza o texto em classes predefinidas com base em seu conteúdo. Por exemplo, detecção de spam e classificação de tópicos.

Extração de informações

Quando é possível, extrai informações específicas, como nomes, datas, locais ou relações não triviais.

Modelagem de linguagem

Captura os padrões estruturais da linguagem para gerar texto coerente e relevante com base no contexto. Por exemplo, uma análise da macroeconomia de determinado segmento de mercado de uma região do Brasil.

Tradução automática

Traduz texto ou fala de um idioma para outro, considerando expressões formais e coloquiais, de modo que o resultado seja coerente para a língua destino.

Respostas a perguntas

Muito utilizado por robôs de chat que conseguem entender o que o usuário está procurando e podem responder perguntas com base em documentos de texto ou bases de conhecimento.

Resumo de texto

Produz resumos essenciais que tragam informações relevantes sobre documentos ou artigos de texto longos.

Redes neurais e aprendizado profundo

Usa modelos computacionais que simulam o comportamento do cérebro com o objetivo de modelar e processar a linguagem.

Normalmente, utilizamos essas técnicas combinadas de forma sequencial. Caso contrário, os algoritmos podem cometer muitos erros, o que gera muito trabalho para corrigir os resultados obtidos. A ideia de usar PLN é trabalhar com escala e ter informações relevantes para suporte à tomada de decisão.

Alguns dos principais erros cometidos pelo PLN

A linguagem é uma das características mais importantes da humanidade. É através dela que podemos expressar ideias, sentimentos, descrições e muitas outras situações úteis. É bem óbvio que a linguagem é muito flexível e isso é um grande desafio para as técnicas de PLN, por isso elas podem cometer erros em determinados cenários. A seguir, vamos listar alguns desses erros cometidos pelas técnicas de PNL.

1

Ambiguidade

Corresponde a uma característica muito comum na linguagem natural, e é claro que gera grandes desafios para os modelos de PLN. É por isso que o contexto é tão importante para determinar a classificação de um texto.

2

Entendimento contextual

As técnicas de PLN utilizam o contexto local para interpretar o significado de palavras ou frases com o objetivo de aumentar a qualidade das respostas. No entanto, contextos muito amplos podem levar a interpretações errôneas ou previsões incorretas.

3

Erros de sintaxe e gramática

Situação que está muito relacionada a contextos confusos que contêm erros de sintaxe ou gramática. É claro que é bem mais desafiante para um algoritmo de PLN capturar informações úteis, no entanto, é um cenário que ocorre com bastante frequência e que, se não tiver o tratamento adequado, pode levar a interpretações erradas.

4

Ironia e sarcasmo

Identificar e interpretar ironia ou sarcasmo depende de dicas explícitas de base de conhecimento de linguagem. Uma mesma expressão pode ter um significado completamente diferente dependendo do contexto, mas não é simples estabelecer até que ponto o algoritmo precisa avaliar um contexto para obter uma resposta correta.

5

Viés e insuficiência de dados

Representa um problema que tem impacto em quaisquer algoritmos de IA. Se os dados utilizados para treinar um modelo forem tendenciosos ou insuficientes, os modelos não vão produzir resultados que possam ser generalizados e, portanto, isso reduz bastante a utilidade prática do modelo.

É natural que existam tantos desafios relacionados às técnicas de PLN, afinal, estamos tratando de uma das características mais difíceis do ser humano: a comunicação.

Atividade 1

Vimos que existem muitas técnicas relacionadas ao PLN. Portanto, somos nós que devemos escolher qual a técnica ou o conjunto de técnicas mais adequado para tratar determinada situação. Também vimos que existem desafios inerentes a essa área. Nesse sentido, selecione a opção que justifica a necessidade de usar PLN em se tratando de situações muito específicas, como o linguajar dos mecânicos de uma oficina de equipamentos pesados.

A

Monitorar a satisfação dos mecânicos com a empresa.

B

Corrigir a forma como os mecânicos se comunicam entre si.

C

Aumentar a quantidade de informações geradas.

D

Coletar dados específicos sobre procedimentos e peças.

E

Eliminar o uso de palavras inadequadas nos textos.



A alternativa D está correta.

Especialmente em locais com linguajar específicos, é muito interessante fazer uso de técnicas de PLN. No caso específico da oficina, podemos obter dados sobre quais peças foram utilizadas e em quais procedimentos, o que é bastante útil para obter informações que podem ser utilizadas de modo quantitativo e, assim, oferecer suporte à tomada de decisões.

Vetorização de palavras

Assista ao vídeo e aprenda sobre a vetorização de palavras como uma técnica de PLN usada para representar palavras ou texto em um formato vetorial numérico.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição de vetorização de palavras

A vetorização de palavras é uma técnica de PLN usada para representar palavras ou texto em um formato vetorial numérico. O objetivo dessas representações vetoriais é capturar as relações semânticas e sintáticas entre as palavras, de modo que os modelos possam trabalhar com números.

Um exemplo de vetorização de palavras

Uma das formas de fazermos vetorização de palavras é através de uma técnica conhecida como Bag-of-Words (na tradução literal, saco de palavras). Vamos ver um exemplo prático através de uma frase de referência.

Frase de entrada: “Eu estudo o processamento de linguagem natural”

Acompanhe os passos para realizar a vetorização usando Bag-of-words!

- **Tokenização:** Divisão da frase em palavras ou tokens individuais:
 - Tokens: ["eu", "estudo", "o", "processamento", "de", "linguagem", "natural"]
- **Criação de vocabulário:** Criar um vocabulário através de tokens exclusivos da frase:
 - Vocabulário: ["eu", "estudar", "processamento", "linguagem", "natural"]
- **Codificação:** A ideia é usar valores binários para representar as palavras. Por exemplo, vamos fazer a representação vetorizada da frase de referência:
 - “Eu” → [1, 0, 0, 0, 0, 0]
 - “estudo” → [0, 1, 0, 0, 0, 0]
 - “processamento” → [0, 0, 1, 0, 0, 0]
 - “de” → [0, 0, 0, 1, 0, 0]
 - “linguagem” → [0, 0, 0, 0, 1, 0]
 - “natural” → [0, 0, 0, 0, 0, 1]
- **Combinação de vetores:** Basicamente, devemos combinar os vetores de palavras individuais para criar uma representação vetorial de toda a frase. No caso do nosso exemplo, basta somar os vetores codificados one-hot: Frase vetorizada: [1, 1, 1, 1, 1, 1]

Nesse caso, fizemos uma abordagem simples usando Bag-of-Words. É um fato que há técnicas mais avançadas, no entanto, o nosso objetivo era dar uma visão geral como podemos obter a vetorização de palavras em números.

Algumas técnicas para vetorização de palavras

Agora, vamos apresentar algumas técnicas populares de vetorização de palavras.

1

Word2Vec

É um modelo de incorporação de palavras. Ele tem como objetivo fazer as incorporações de palavras que capturam relações semânticas e utiliza dois algoritmos principais que vamos tratar com mais detalhes: Continuous Bag of Words (CBOW) e Skip-Gram.

2

GloVe

Corresponde à sigla para Global Vectors for Word Representation que, traduzido para o português, fica como Vetorização Global para Representação de Palavras. O objetivo desta técnica é capturar estatísticas globais em diversos textos e, a partir disso, ser útil para aplicações de representações significativas que podem capturar relações semânticas e sintáticas de frases locais.

FastText

É uma extensão do Word2Vec que representa palavras como um pacote de n-gramas de caracteres. Um n-grama é simplesmente uma combinação de “n” palavras. No caso do exemplo de referência, aqui estão algumas situações “2-gramas”: (“eu”, “estudo”); (“processamento”, “linguagem”). Há casos que essa combinação é bastante interessante e pode fornecer informações valiosas sobre o texto.

4

BERT

É uma técnica que faz as representações de codificador bidirecional de transformadores. É um modelo de linguagem muito recente que já é pré-treinado. Ele gera incorporações de palavras contextualizadas, considerando o contexto esquerdo e direito de cada palavra.

5

Modelos baseados em transformadores

São modelos muito modernos conhecidos pela sigla GPT (transformador pré-treinado generativo) e GPT-2. Esses modelos utilizam mecanismos avançados para gerar incorporações de palavras contextualizadas, capturando dependências de longo alcance e informações contextuais de forma eficaz.

Quando avançamos nos trabalhos com PLN, é natural usarmos alguma dessas técnicas de vetorização de palavras, especialmente quando utilizamos algoritmos de aprendizado de máquina. A vetorização é um mecanismo muito útil, pois permite utilizarmos técnicas de estatística e álgebra linear para manipular os dados e, dessa forma, fazermos classificação de texto, recuperação de informações, análise de sentimento e tradução automática.

Atividade 2

A matéria-prima das técnicas de PLN são textos de palavras. No entanto, vimos que existem diversas técnicas que têm como objetivo transformar esses textos em vetores de números. Nesse sentido, selecione a opção correta que contém uma justificativa adequada para aplicar a vetorização em textos.

A

Com vetores numéricos, podemos aplicar técnicas bem consolidadas das áreas de exatas.

B

É mais fácil manipular números do que palavras.

C

Palavras ocupam muito espaço na memória e isso torna o processamento ineficiente.

D

Os vetores numéricos são usados para ordenar as palavras mais importantes.

E

Através de vetores de números, podemos realizar operações equivalentes ao que faríamos com palavras.



A alternativa A está correta.

A vetorização é importante, pois os modelos computacionais de PLN utilizam números para realizar cálculos. Portanto, temos mais recursos para utilizar técnicas de matemática e estatística para obter informações e, em alguns casos, relações relevantes que, de outro modo, seria muito difícil.

Visão geral da CBOW

Assista ao vídeo e confira uma introdução sobre a técnica CBOW. Ela é bastante utilizada em PLN exatamente por ter demonstrado bons resultados práticos.

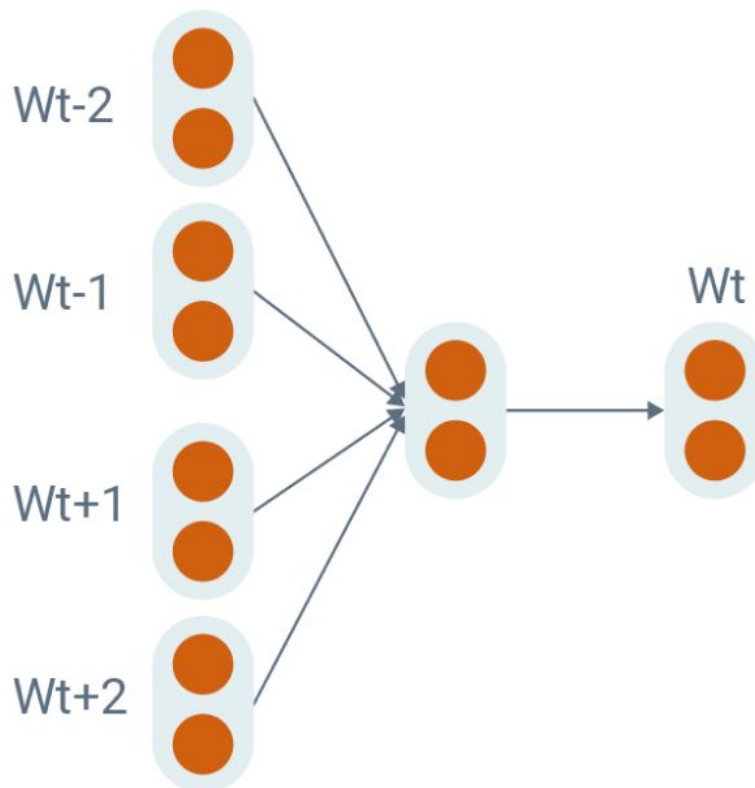


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição da técnica Continuous Bag of Words (CBOW)

É uma técnica que faz a incorporação de palavras através da combinação delas, conforme o contexto. Sendo mais detalhista, ela é um modelo computacional que aprende a prever uma palavra-alvo com base nas palavras do contexto do entorno. A seguir, confira uma imagem que nos dá uma ideia de como esse modelo trabalha.



Exemplo de CBOW

A ideia básica do modelo CBOW é gerar representações vetoriais densas que capturam relações semânticas entre palavras. Em outras palavras, dado um conjunto de palavras, ele propõe a palavra mais adequada para compor o conjunto.

Agora, vamos analisar algumas vantagens e desvantagens da técnica CBOW.

Vantagens CBOW

Confira as vantagens do modelo CBOW!

1

Eficiência

É uma técnica eficiente em comparação a outras técnicas de incorporação de palavras como Skip-Gram. Além disso, devido à arquitetura mais simples, ele é adequado para conjuntos de dados de grande escala.

2

Generalização

Pode lidar bem com palavras fora do vocabulário, usando, para isso, o contexto em que elas aparecem.

3

Treinamento com pequenos conjuntos de dados

Funciona relativamente bem, mesmo com dados de treinamento limitados.

Desvantagens CBOW

Agora, conheça as desvantagens do modelo CBOW!

1

Perda da ordem das palavras

Trata as palavras de entrada como um conjunto não ordenado, ignorando sua ordem sequencial. Dessa forma, pode não capturar as dependências de ordem de palavras que são importantes em algumas tarefas de PLN.

2

Janela de contexto limitada

Considera apenas uma janela de contexto de tamanho fixo em torno de cada palavra. Dessa forma, ele pode não capturar dependências de longo alcance ou informações contextuais fora da janela.

3

Sensibilidade aos dados de treinamento

Depende fortemente da qualidade e distribuição dos dados de treinamento. Ou seja, dados de treinamento tendenciosos ou não representativos podem levar a incorporações de palavras tendenciosas ou imprecisas e, assim, potencialmente pode propagar vieses.

4

Falta de representação de palavras raras

Não trata bem palavras que ocorrem com pouca frequência nos dados de treinamento.

O fato é que o CBOW é uma técnica muito usada na prática e é importante que tenhamos noção de como ela funciona e quais são suas vantagens e desvantagens.

Funcionamento da técnica CBOW

Vejamos as etapas do funcionamento básico da técnica CBOW!

Etapa 1

Preparação do corpus: Os dados de treinamento são pré-processados pela tokenização do texto em palavras individuais e usados para criação de um vocabulário de termos exclusivos.

Etapa 2

Janela de contexto: Para cada palavra do corpus (conjunto de palavras), é definida uma janela de contexto de tamanho fixo. Essa janela de contexto determina o número de palavras antes e depois da palavra-alvo que será considerada como entrada.

Etapa 3

Geração de pares entrada-saída: Com os dados de treinamento, pares entrada-saída são gerados, sendo que a entrada consiste nas palavras de contexto dentro da janela definida e a saída é a palavra de destino. Por exemplo, dada a frase “eu estudo processamento de linguagem natural”, os pares de entrada-saída seriam:

- Entrada: ["Eu", "processamento"]
- Saída: “estudo”

Etapa 4

Arquitetura de rede neural: O CBOW usa um modelo computacional chamado de rede neural feedforward simples com uma única camada oculta, cujo objetivo é incorporar as palavras de contexto através de uma transformação em uma representação vetorial.

Etapa 5

Treinamento: O modelo CBOW é treinado usando os pares de entrada-saída gerados. Dessa forma, o modelo tem como meta prever a palavra-alvo com base nas palavras do contexto de entrada.

Etapa 6

Geração de incorporação de palavras: Depois que o modelo CBOW é treinado, ele incorpora as palavras e está pronto para ser usado.

O principal uso da técnica CBOW é para capturar o contexto de uma palavra considerando as palavras do contexto. Normalmente, ele funciona bem com palavras frequentes e é rápido para realizar treinamento de outros modelos de aprendizado.

Atividade 3

Uma das técnicas de PLN é o CBOW. Um dos elementos básicos dessa técnica é a vetorização de palavras. Nesse sentido, selecione a opção correta que contém a justificativa adequada de quando devemos usar o CBOW.

A

Deve ser utilizado como um modelo de PLN para fazer classificações exatas de palavras.

B

Deve ser usado sempre que pretendermos obter diversas palavras com significados semânticos semelhantes.

C

Essa técnica funciona apenas quando existem poucas palavras para treinar o modelo.

D

Deve ser usado como um modelo auxiliar para estimar palavras dado determinado contexto.

E

É o modelo mais eficiente de PLN para qualquer situação de classificação de palavras.



A alternativa D está correta.

O CBOW é uma técnica muito utilizada nos métodos de PLN para estimar palavras dado determinado contexto. Ele possui vantagens e desvantagens que precisam ser levados em consideração, quando for escolhido para incorporar um modelo de processamento de linguagem natural.

Skip-Gram

Assista ao vídeo e confira uma introdução sobre a técnica de PLN Skip-Gram, além de analisar suas vantagens e desvantagens.



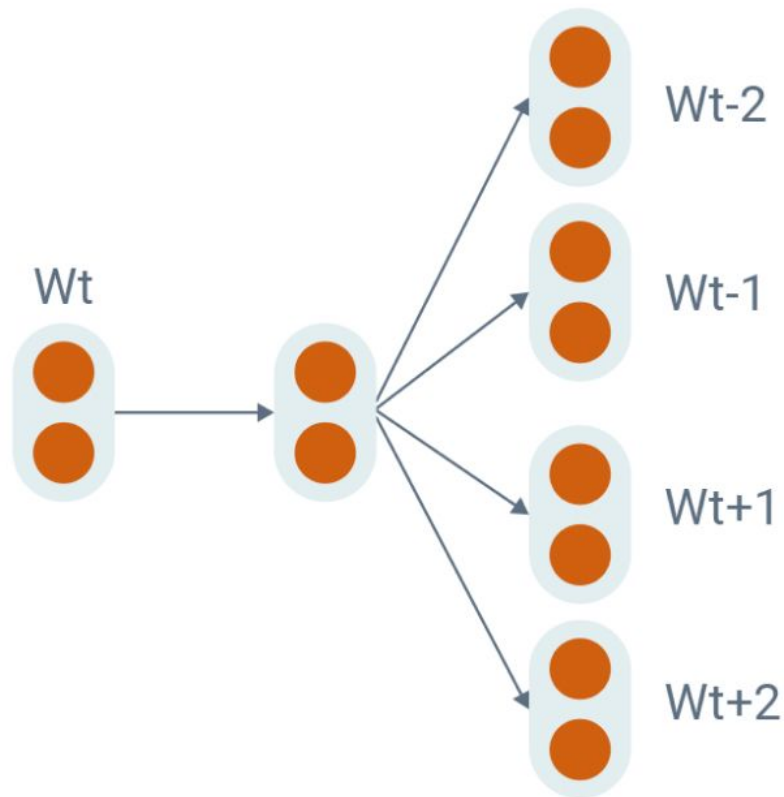
Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição da técnica Skip-Gram

A técnica Skip-Gram é mais um método de incorporação de palavras usado em PLN. Diferente da técnica CBOW, que estima a palavra-alvo em relação a um contexto, o Skip-Gram tem como objetivo prever as palavras do contexto dada uma palavra-alvo. Ela aprende como funcionam as incorporações das palavras através das relações semânticas entre elas. Ou seja, as palavras que ela vai estimar não são escolhas aleatórias. Elas são o resultado de um processo de treinamento que captura qual o uso mais adequado de determinada palavra-alvo em relação a um contexto estimado.

A seguir, apresentamos uma representação de como o modelo funciona.



Exemplo do Skip-Gram

Basicamente, a técnica Skip-Gram segue as etapas:

Etapa 1

Preparação do corpus (conjunto de palavras): É semelhante ao CBOW, ou seja, os dados de treinamento vêm de grandes conjuntos de texto que são pré-processados pela tokenização do texto em palavras e usados na criação de um vocabulário de termos exclusivos.

Etapa 2

Definição do tamanho da janela: Para cada palavra do corpus (conjunto de palavras que fazem parte do modelo), definimos uma janela de tamanho fixo. É essa janela que determina o número de palavras antes e depois da palavra-alvo que será considerada como o contexto.

Etapa 3

Geração de pares entrada-saída: Após o treinamento dos dados, o modelo vai gerar pares entrada-saída. A palavra de destino serve como entrada e as palavras de contexto dentro da janela definida atuam como saídas. Por exemplo, vamos considerar a frase: “Eu estudo processamento de linguagem natural” com um tamanho de janela de 2. Os pares de entrada-saída seriam:

- **Entrada:** “processamento”
- **Saída:** [“estudo”, “de”]
- **Entrada:** “linguagem”
- **Saída:** [“de”, “natural”]

Etapa 4

Arquitetura do modelo: O Skip-Gram utiliza um modelo computacional conhecido como rede neural. A ideia é bem similar ao que ocorre com o CBOW. Esse modelo incorpora o relacionamento entre as palavras e, assim, define a semântica delas.

Etapa 5

Treinamento: Novamente, o modelo se assemelha ao CBOW no sentido de que ele é treinado através dos pares de entrada-saída gerados. O processo de treinamento tem como objetivo realizar ajustes para minimizar o erro de previsão.

Etapa 6

Geração de incorporação de palavras: Depois de treinado, o modelo passa a incorporar a semântica das palavras.

A técnica Skip-Gram é particularmente eficaz em situações que palavras ocorrem com pouca frequência. As aplicações são as mesmas do CBOW, apesar de ambas as técnicas apresentarem uma forma muito diferente de trabalhar.

Vantagens e desvantagens da Skip-Gram

Agora, vamos explorar um pouco os pontos positivos e negativos da técnica Skip-Gram no processamento de linguagem natural.

Vantagens da Skip-Gram

Confira agora quais são as vantagens da técnica Skip-Gram.

1

Captura de relacionamentos de palavras

Pode gerar incorporações de palavras que refletem o significado e o contexto delas.

2 Flexibilidade contextual

Permite prever várias palavras de contexto a partir de uma única palavra-alvo. Dessa forma, ele pode capturar diversas associações e dependências de palavras.

3

Representação de palavras raras

Pode gerar incorporações significativas para palavras de baixa frequência.

4

Similaridade entre as palavras

Pode capturar fortes semelhanças entre as palavras, tanto nas estruturas sintática como semântica.

5

Transferência de aprendizado

Apresenta uma base de conhecimento formada por um modelo que pode ser incorporada a outros modelos.

Desvantagens da Skip-Gram

Agora, vamos falar um pouco sobre as desvantagens da Skip-Gram. Entre elas estão:

1

Complexidade computacional

Consome mais recursos e tempo em comparação com técnicas, como CBOW.

2

Falta de preservação da ordem das palavras

Trata as palavras como entidades independentes e não considera explicitamente sua ordem sequencial.

3

Sensibilidade aos dados de treinamento

Apresenta um impacto significativo no desempenho do modelo.

4

Maiores requisitos de dados de treinamento

Para alcançar um bom desempenho, o modelo demanda por mais dados de treinamento em comparação com o CBOW.

Semelhante ao que fizemos com o CBOW, precisamos considerar as vantagens e desvantagens ao decidir usar o Skip-Gram.

Atividade 4

Entre as técnicas de processamento de linguagem natural, temos o Skip-Gram. Apesar de ter um processo similar ao CBOW, ambas as técnicas são bem diferentes. Nesse sentido, selecione a opção correta sobre a vantagem de usar o Skip-Gram em um editor de textos.

A

Sugestões de textos completos.

B

Sugestões de palavras.

C

Possibilidade de criar perguntas e respostas facilmente.

D

Eliminar preocupações com autoria dos textos.

E

Aumentar as chances de usar palavras muito sofisticadas no texto.



A alternativa B está correta.

Certamente, há muitas situações vantajosas para utilizar o método Skip-Gram. No caso de editores de texto, o algoritmo pode propor palavras que sejam adequadas ao contexto que estamos escrevendo, o que é bastante útil para produzirmos conteúdos com maior produtividade.

A biblioteca regex

Assista ao vídeo e tenha uma visão geral da biblioteca regex, além de aprender como utilizar as funcionalidades e metacaracteres para obter resultados muito interessantes.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Expressões regulares (regex) são uma ferramenta poderosa para trabalhar com texto e podem ser aplicadas em diversas tarefas de PLN. Através da regex, podemos definir padrões e manipular dados de textos. Conheça as aplicações em que podemos usar regex no contexto de PLN.

1 Correspondência de padrões

Podemos definir padrões para pesquisar sequências específicas de caracteres no texto. Por exemplo, pode localizar todas as ocorrências de uma palavra específica ou um padrão específico, como endereços de e-mail, URLs ou datas em um documento de texto.

2

Tokenização

É uma etapa básica dos métodos de PLN.

3

Limpeza de texto

É bastante útil para realizarmos pré-processamentos e limpeza dos dados de texto.

4

Extração de padrão

É útil para nos ajudar a extrair informações específicas do texto, como nomes de pessoas, placas de carro, CEPs de ruas, por exemplo.

5

Validação de texto

Usamos para validar se o formato das entradas de texto corresponde a um padrão específico. Por exemplo, podemos usar regex para verificar se uma entrada corresponde a um padrão válido de endereço de e-mail válido ou um número de telefone.

De um modo geral, regex é muito útil e, por isso mesmo, é bastante usada nas primeiras etapas dos algoritmos de PLN. A seguir, vamos conhecer características mais específicas da biblioteca como funções e metacaracteres.

Funções e metacaracteres

A biblioteca regex permite que façamos uso de algumas funções para manipular os dados através de padrões. Esses padrões, por sua vez, são representados por combinações de símbolos especiais chamados de metacaracteres. A seguir, vamos conhecer como eles funcionam com mais detalhes.

Funções

A biblioteca regex está disponível para diversas linguagens de programação. No nosso caso, vamos utilizar a linguagem Python que utiliza o módulo "re" para fazer referência às funcionalidades da biblioteca regex. Na sequência, apresentamos quais são essas funções e uma breve descrição de cada uma delas.

findall

Retorna uma lista contendo todas as correspondências das palavras que satisfazem determinado padrão.

search

Retorna um objeto do tipo “match”, caso haja alguma correspondência em qualquer lugar do texto.

split

Retorna uma lista em que o texto é separado de acordo com algum critério. É muito utilizada para fazer tokenização.

sub

É usada para substituir uma ou várias correspondências por uma sequência.

A seguir, vamos conhecer os metacaracteres.

Metacaracteres

São simplesmente caracteres com um significado especial. Eles são úteis para descrever padrões. Confira agora os principais metacaracteres da biblioteca regex e uma breve descrição sobre eles.

[]

Um conjunto de caracteres.

\

Sinaliza uma sequência especial e pode ser usada para escapar de caracteres especiais.

.

Qualquer caractere (exceto caractere de nova linha).

^

Começa com.

\$

Termina com.

*

Zero ou mais ocorrências.

+

Uma ou mais ocorrências.

{ }

Exatamente o número especificado de ocorrências.

|

Ou.

()

Captura e grupo.

Agora, vamos estudar as sequências especiais que são muito importantes para descrever padrões.

Sequências especiais

São uma combinação do símbolo “\” seguidas por um dos caracteres da lista a seguir. Dessa forma, eles passam a ter um significado especial e são usados para descrever padrões. Vamos conhecer os metacaracteres e uma breve descrição dos seus significados.

`\A`

Retorna uma correspondência se os caracteres especificados estiverem no início da string.

`\b`

Retorna uma correspondência em que os caracteres especificados estão no início ou no final de uma palavra.

`\B`

Retorna uma correspondência em que os caracteres especificados estão presentes, mas não no início (ou no final) de uma palavra.

`\d`

Retorna uma correspondência em que a string contém dígitos (números de 0 a 9).

`\D`

Retorna uma correspondência em que a string não contém dígitos.

`\s`

Retorna uma correspondência em que a string contém um caractere de espaço em branco.

`\S`

Retorna uma correspondência em que a string não contém um caractere de espaço em branco.

`\w`

Retorna uma correspondência em que a string contém caracteres de palavras (caracteres de A a Z, dígitos de 0 a 9 e o caractere sublinhado `_`).

`\W`

Retorna uma correspondência em que a string NÃO contém palavras de caracteres.

`\Z`

Retorna uma correspondência se os caracteres especificados estiverem no final da sequência.

Na sequência, vamos estudar os conjuntos de caracteres.

Conjunto de caracteres

Representa elementos dentro de um par de colchetes `[]` com um significado especial. Na sequência, apresentamos alguns exemplos do uso de conjuntos e quais são os seus significados.

`[arn]`

Retorna uma correspondência em que um dos caracteres especificados (a, r ou n) está presente.

`[a-z]`

Retorna uma correspondência para qualquer caractere minúsculo, em ordem alfabética entre a e z.

`[^arn]`

Retorna uma correspondência para qualquer caractere, exceto a, r e n.

`[0123]`

Retorna uma correspondência em que qualquer um dos dígitos especificados (0, 1, 2 ou 3) esteja presente.

`[0-9]`

Retorna uma correspondência para qualquer dígito entre 0 e 9.

`[0-5] [0-9]`

Retorna uma correspondência para qualquer número de dois dígitos de 0 a 5 e de 0 a 9.

`[a-zA-Z]`

Retorna uma correspondência para qualquer caractere em ordem alfabética entre a e z, minúsculas ou maiúsculas.

`[+]`

Significa que retorna uma correspondência para qualquer caractere “+” na palavra.

Atividade 5

Não há dúvidas sobre a importância prática do processamento de linguagem natural (PLN). Um dos instrumentos muito úteis para realizar esse trabalho são as funcionalidades e metacaracteres da biblioteca regex. Nesse sentido, selecione a opção correta que contém uma justificativa válida para fazermos uso da biblioteca nas tarefas de PLN.

A

Ela disponibiliza recursos que tornam a identificação de padrões mais eficiente.

B

Ela é muito simples de ser usada e possui funções que podem identificar qualquer tipo de padrão

C

Os metacaracteres são muito simples de serem aplicados para manipular palavras em textos.

D

As funções da regex podem ser usadas para substituir técnicas como a CBOW ou Skip-Gram.

E

As funções regex são facilmente adaptadas a qualquer contexto de PLN.



A alternativa A está correta.

Não há dúvidas de que a biblioteca regex disponibiliza recursos muito úteis para PLN, especialmente nas etapas iniciais, em que o nosso objetivo é verificar padrões definidos e fazer limpeza dos dados. Por outro lado, ela não oferece recursos mais elaborados para completar frases ou estimar quais as próximas palavras que estão semanticamente relacionadas a uma palavra especial, como no caso das técnicas CBOW e Skip-Gram, respectivamente.

Tokenização com regex

Assista ao vídeo e aprenda como aplicar a biblioteca regex no Python como um recurso básico para métodos de PLN.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você aplique a técnica de tokenização em um texto através da biblioteca regex usando o Python. No entanto, você deverá selecionar apenas as palavras que possuem acima de determinado comprimento. Para alcançar esse objetivo, siga as seguintes etapas:

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código:

```
plain-text
!pip install regex
```

O objetivo é instalar a biblioteca Regex, para que possamos executar os processamentos de texto.

Passo 4

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
import re
```

O objetivo é importar a biblioteca Regex.

Detalhe muito importante: O aninhamento do código faz parte da sintaxe do Python, então reproduza exatamente o código como está aqui, certo?

Passo 5

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python

texto = "Este é um texto simples que deve ser testado."
```

Passo 6

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python

tokens = re.findall(r'\w{4,}', texto)
```

Você acabou de usar a função “findall” da biblioteca “regex”, que tem como objetivo procurar todos os padrões com palavras que tenham comprimento maior ou igual a 4 (quatro).

Passo 7

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python

print(tokens)
```

O resultado é da execução é dado por:

```
['Este', 'texto', 'simples', 'deve', 'testado']
```

Você acabou de obter uma lista com palavras de comprimento maior ou igual a 4 do texto original e, agora, pode utilizá-las em outras etapas do processamento de linguagem natural. Quase sempre, essa é a primeira parte que devemos executar nesse tipo de trabalho: fazer uma limpeza – outros autores usam o termo “sanitização” – do texto. O objetivo é focar nos termos mais relevantes do texto que nos ajudam a entender qual o significado dele e qual a melhor forma do algoritmo tratar.

Parabéns por ter chegado até aqui! Esse é o caminho muito sólido para aprender, de fato, o que está acontecendo no PLN e não ser apenas um usuário de ferramentas. Agora, é a sua vez de fazer um exercício que vai ajudar você a consolidar seus conhecimentos.

Atividade 1

Uma situação que todos nós usamos com muita frequência na prática são os chamados vícios de linguagem. É óbvio que precisamos ficar muito atentos para evitá-los. Uma forma de nos policiarmos é identificá-los e, nesse sentido, os métodos de PLN podem ser muito úteis. Nesse sentido, implemente um programa em Python utilizando a biblioteca regex para identificar se determinado texto utiliza os termos “né” e “enfim”. Como dica, comece seu código da seguinte forma:

```
python

import re
vícios = ["né", "enfim"]
texto = "Este é um texto, né? Temos que encontrar os vícios de linguagem, enfim."
```

...

Uma situação que todos nós usamos com muita frequência na prática são os chamados vícios de linguagem. É óbvio que precisamos ficar muito atentos para evitá-los. Uma forma de nos policiarmos é identificá-los e,

nesse sentido, os métodos de PLN podem ser muito úteis. Nesse sentido, implemente um programa em Python utilizando a biblioteca regex para identificar se determinado texto utiliza os termos “né” e “enfim”. Como dica, comece seu código da seguinte forma:

Chave de resposta

Há diversas formas de resolver esse problema. Por exemplo, você pode utilizar o seguinte código:

```
python

import re
vicios = ["né", "enfim"]
texto = "Este é um texto, né? Temos que encontrar os vícios de linguagem, enfim."
palavras_encontradas=[]
for padrao in vicios:
    if re.search(padrao, texto):
        palavras_encontradas.append(padrao)
print(f'O total de palavras encontradas é de :{len(palavras_encontradas)}')
```

Cuja resposta é:

O total de palavras encontradas é de: 2.

Limpeza de texto com regex

Assista ao vídeo e conheça com mais profundidade os recursos da biblioteca regex do Python para trabalhar com PLN.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você elimine determinadas palavras de um texto. Nos primeiros passos do PLN, precisamos focar o máximo possível na informação que é essencial para o entendimento do contexto que estamos trabalhando. Novamente, vamos utilizar a biblioteca regex usando o Python. A partir daqui, já supomos que você instalou a biblioteca com sucesso; caso contrário, o programa não vai funcionar. Para alcançar esse objetivo, siga as seguintes etapas:

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código:


```
python
import re
```

Acabamos de importar a biblioteca regex.

Passo 4

Agora, crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
texto = "Este texto é um exemplo para fazer um checklist de revisão do motor do carro."
palavras_para_eliminar = ["este", "para", "de"]
texto = texto.lower()
```

Trata-se do texto que vamos manipular e das palavras que queremos eliminar. Além disso, transformamos o texto original para letras minúsculas.

Passo 5

Agora, crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
for palavra in palavras_para_eliminar:
    padrao = r'\b%s\b' % re.escape(palavra)
    texto = re.sub(padrao, '', texto)
```

É aqui que ocorre o processamento do texto. Verificamos cada uma das palavras que devem ser eliminadas e, quando as encontramos no texto, fazemos a substituição por um símbolo vazio.

Passo 6

Crie outra célula de código. Escreva e execute, exatamente, o seguinte código:

```
python
print(f'O resultado é: {texto}')
```

Cuja resposta é dada por:

O resultado é: texto é um exemplo fazer um checklist revisão do motor do carro.

Ou seja, as palavras “este”, “para” e “de” foram eliminadas. Agora, é sua vez de praticar e avançar mais um pouco nessa área tão promissora. Bons estudos!

Atividade 2

No tratamento de texto é comum precisarmos eliminar determinadas palavras, mas, além disso, também é natural termos que eliminar palavras muito curtas. Nesse sentido, aproveite parte do código que você acabou de estudar e desenvolva uma solução em Python usando a biblioteca regex que, além de eliminar determinadas palavras, também elimine as que possuem comprimento menor ou igual a 2. Como dica para construir a sua solução, utilize os metacaracteres “\b” e “\w” e o operador “|”.

Chave de resposta

Uma proposta de solução deve ter os seguintes elementos:

```
python

import re
texto = "Este texto é um exemplo para fazer um checklist de revisão do motor do carro."
palavras_para_eliminar = ["este", "para", "de"]
texto = texto.lower()
for palavra in palavras_para_eliminar:
    padrao = r'\b%s\b' % re.escape(palavra)
    padrao_palavras_curtas = r'\b\w{1,2}\b'
    padroes_combinados = f'{padrao}|{padrao_palavras_curtas}'
    texto = re.sub(padroes_combinados, '', texto)

print(f'Resultado: {texto}')
```

Cujo resultado é:

Resultado: texto exemplo fazer checklist revisão motor carro.'

Perceba que eliminamos as palavras “Este”, “é”, “um”, “para”, “de” e “do”. Para isso, tivemos que aperfeiçoar a nossa expressão regex.

Construção de dicionários

Assista ao vídeo e aprenda a trabalhar com dicionários no Python, entendendo como isso será útil para PLN.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é que você construa um dicionário de palavras. No final, o programa deverá informar o total de palavras. Essa é mais uma situação bastante comum em PLN: trabalharmos com dicionários. Aqui, vamos utilizar o pacote pandas do Python. Agora, vamos seguir as seguintes etapas:

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código:

```
python  
import pandas
```

Passo 4

Escreva e execute, exatamente, o seguinte código:

```
python  
dicionario = {'fruta': ['maçã', 'banana', 'laranja'],  
              'animal': ['cachorro', 'leão', 'peixe'],  
              'veiculo': ['carro', 'moto', 'onibus']}
```

Você acabou de criar um dicionário de palavras. Agora, vamos adicionar esse dicionário para um Dataframe.

Passo 5

Escreva e execute, exatamente, o seguinte código:

```
python  
df = pd.DataFrame(dicionario)
```

Passo 6

Na sequência, vamos criar uma lista com o Dataframe. Escreva e execute, exatamente, o seguinte código:

```
python  
palavras_dicionario = df.to_dict('list')
```

Passo 7

Na sequência, vamos contar a quantidade de vezes que cada item aparece. Escreva e execute, exatamente, o seguinte código:

```
python  
qtd_item = {}  
for key, values in palavras_dicionario.items():  
    for valor in values:  
        qtd_item[valor] = qtd_item.get(valor, 0) + 1
```

Passo 8

Por fim, vamos exibir o item e a quantidade de vezes que ele aparece. Escreva e execute, exatamente, o seguinte código:

```
python

print("Quantidade de Itens:")
for item, qtd in qtd_item.items():
    print(f"{item}: {qtd}")
```

A resposta do programa é:

Quantidade de Itens:

maçã: 1
banana: 1
laranja: 1
cachorro: 1
leão: 1
peixe: 1
carro: 1
moto: 1
onibus: 1

Bem, agora chegou a hora de você colocar seus conhecimentos em prática. Bons estudos!

Atividade 3

Na prática anterior, o nosso objetivo foi construir um dicionário e contar a quantidade de palavras presente nele. Agora, precisamos construir um programa que receba um texto e um dicionário e apresente quantas vezes cada categoria das palavras do dicionário apareceu no texto.

Como sugestão, inicie seu programa com o seguinte código:

```
python

import pandas as pd
import re

dicionario = {'fruta': ['maçã', 'banana', 'laranja'],
              'animal': ['cachorro', 'leão', 'peixe'],
              'veiculo': ['carro', 'moto', 'onibus']}

texto = "meu cachorro gosta de passear de carro e de comer maçã."
padrao_sem_pontuacao = r'^\w\s]'
texto = re.sub(padrao_sem_pontuacao, '', texto)

contagem_categorias = {key: 0 for key in dicionario.keys()}
palavras = texto.lower().split()

## continuar a partir desse trecho.
```

Em seguida, use a criatividade e a capacidade de pesquisar para concluí-lo.

Chave de resposta

O programa deve ficar com o seguinte aspecto:

```
python

import pandas as pd
import re

dicionario = {'fruta': ['maçã', 'banana', 'laranja'],
              'animal': ['cachorro', 'leão', 'peixe'],
              'veiculo': ['carro', 'moto', 'onibus']}

texto = "meu cachorro gosta de passear de carro e de comer maçã."
padrao_sem_pontuacao = r'^\w\s$'
texto = re.sub(padrao_sem_pontuacao, '', texto)

contagem_categorias = {key: 0 for key in dicionario.keys()}
palavras = texto.lower().split()

for palavra in palavras:
    for categoria, sinonimos in dicionario.items():
        if palavra in sinonimos:
            contagem_categorias[categoria] += 1

print(contagem_categorias)
```

Resposta:

```
{'fruta': 1, 'animal': 1, 'veiculo': 1}
```

Perceba que exploramos a biblioteca regex e as propriedades do dicionário para obtermos um código mais legível. No entanto, existem outras formas de chegar ao mesmo resultado. O importante é que você tenha domínio do que está trabalhando e como isso realmente possa ser útil.

Extração de datas

Assista ao vídeo e aprenda como usar alguns recursos da biblioteca regex para extrair datas de um texto-livre.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é construirmos um código que extraia dias e meses de um texto. No final, nosso programa deverá informar uma lista com uma separação de dias e meses que encontrou ao longo de todo o texto. É bem comum precisarmos obter esse tipo de informação como um elemento de suporte à tomada de decisão. Mais uma vez, vamos resolver esse problema com o uso da biblioteca regex do Python. Agora, vamos seguir as seguintes etapas:

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código:

```
python
import re
```

Passo 4

Escreva e execute, exatamente, o seguinte código:

```
python
texto = "Existem várias datas comemorativas, como o natal no dia 25/12 e 1/1"
```

Você acabou de criar o texto que vamos usar para extrair as datas. Agora, vamos criar nosso padrão de extração de data.

Passo 5

Escreva e execute, exatamente, o seguinte código:

```
python
padrao = r'\b(\d{1,2})/(\d{1,2})\b'
```

Passo 6

Agora, vamos procurar todas as combinações do padrão no texto. Escreva e execute, exatamente, o seguinte código:

```
python
combinacoes = re.findall(padrao, texto)
```

Passo 7

Agora, precisamos apenas extrair os padrões e adicioná-los a uma lista. Escreva e execute, exatamente, o seguinte código:

```
python
datas = []
for d in combinacoes:
    dia = int(d[0])
    mes = int(d[1])
    datas.append((dia, mes))
```

Passo 8

Por fim, vamos exibir os resultados obtidos. Escreva e execute exatamente o seguinte código:

```
python  
print(datas)
```

Cujo resultado é:

```
[(25, 12), (1, 1)]
```

Agora, é sua vez de praticar.

Atividade 4

Utilize o programa anterior para obter também o ano de cada data. Ou seja, o programa deve procurar pelo padrão: dd/mm/aaaa.

Sugestão: utilize o seguinte texto:

```
python  
  
texto = "No dia 07/07/1822, foi proclamada a independência do Brasil e no dia 15/11/1899,  
foi proclamada a república no Brasil"
```

Chave de resposta

Basta adicionar mais um elemento padrão da expressão regular para extrair o ano. O código vai ficar assim:

```
python  
  
import re  
texto = "No dia 07/07/1822, foi proclamada a independência do Brasil e no dia  
15/11/1899, foi proclamada a república no Brasil"  
# Expressão Regular para combinar o formato de datas {d}d/{m}m/aaaa  
padrao = r'\b(\d{1,2})/(\d{1,2})/(\d{4})\b'  
  
# Encontrar todas as datas no texto  
combinacoes = re.findall(padrao, texto)  
  
# Process the matches  
datas = []  
for d in combinacoes:  
    dia = int(d[0])  
    mes = int(d[1])  
    ano = int(d[2])  
    datas.append((dia, mes, ano))  
  
print(datas)
```

Cuja resposta é: [(7, 7, 1822), (15, 11, 1899)]

Análise de sentimentos na prática

Assista ao vídeo e aprenda a utilizar recursos de PLN para extrair o sentimento de um texto-livre com o uso da biblioteca regex.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

O objetivo dessa prática é construirmos um código que faça a análise de sentimento de um texto-livre. No final, nosso programa deverá informar se o sentimento do texto é positivo, negativo ou neutro, conforme os critérios que vamos utilizar. Essa é uma das principais aplicações de PLN, pois auxilia a entender a reação do público sobre determinado produto ou serviço e, assim, gerando oportunidades aos tomadores de decisão a direcionarem suas energias para o local correto. Mais uma vez, vamos resolver esse problema com o uso da biblioteca regex do Python. Agora, vamos seguir as seguintes etapas!

Passo 1

Crie um projeto novo no Google Colab.

Passo 2

Crie uma célula de código.

Passo 3

Escreva e execute, exatamente, o seguinte código:

```
python
import re
```

Passo 4

Escreva e execute, exatamente, o seguinte código:

```
python
palavras_positivas = ['bom', 'excelente', 'feliz']
palavras_negativas = ['ruim', 'péssimo', 'triste']
```

Você acabou de criar as listas de palavras positivas e negativas que vamos usar para analisar o texto.

Passo 5

Agora, vamos entrar com o texto para testar. Escreva e execute, exatamente, o seguinte código:

```
python
texto = "O filme foi excelente. De fato, foi muito bom."
```

Passo 6

A próxima tarefa é contar a quantidade de palavras positivas e negativas que a regex vai procurar no texto. Para isso, escreva e execute, exatamente, as seguintes linhas de código:


```
python
```

```
qtd_positiva = len(re.findall(r'\b(?:%s)\b' % '|'.join(palavras_positivas), texto, flags=re.IGNORECASE))
qtd_negativa = len(re.findall(r'\b(?:%s)\b' % '|'.join(palavras_negativas), texto, flags=re.IGNORECASE))
```

Passo 7

Agora, precisamos apenas comparar qual a quantidade predominante entre os termos positivos e negativos. Escreva e execute, exatamente, o seguinte código:

```
python
```

```
sentimento='Neutro'
if qtd_positiva > qtd_negativa:
    sentimento='Positivo'
elif qtd_negativa > qtd_positiva:
    sentimento='Negativo'
```

Passo 8

Por fim, vamos exibir os resultados obtidos. Escreva e execute, exatamente, o seguinte código:

```
python
```

```
print(f'O sentimento sobre o filme foi: {sentimento}')
```

Cujo resultado é:

O sentimento sobre o filme foi: Positivo

Agora, é sua vez de praticar!

Atividade 5

Desenvolva um programa que determine o sentimento de um texto-livre com base no seguinte critério: se a quantidade de palavras positivas for igual ou maior que o dobro de palavras negativas, então o sentimento é positivo. Se as palavras negativas forem o triplo de palavras positivas, então o sentimento é negativo. Caso contrário, o sentimento é neutro.

Dica: utilize o seguinte trecho de código para iniciar a sua solução.

```
python
```

```
import re
```

```
palavras_positivas = ['bom', 'excelente', 'feliz']
palavras_negativas = ['ruim', 'péssimo', 'triste']
```

```
texto = "O filme foi excelente. De fato, foi muito bom, mas teve um final triste."
```

```
# Continuar daqui
```

Chave de resposta

Uma possível solução para o programa é:

```
python

import re

palavras_positivas = ['bom', 'excelente', 'feliz']
palavras_negativas = ['ruim', 'péssimo', 'triste']

texto = "O filme foi excelente. De fato, foi muito bom, mas teve um final triste."

# Conte as ocorrências de palavras positivas e negativas no texto
qtd_positiva = len(re.findall(r'\b(?:%s)\b' % '|'.join(palavras_positivas),
                             texto, flags=re.IGNORECASE))
qtd_negativa = len(re.findall(r'\b(?:%s)\b' % '|'.join(palavras_negativas),
                             texto, flags=re.IGNORECASE))

# Determine o sentimento com base na contagem de palavras
sentimento='Neutro'
if qtd_positiva > 2*qtd_negativa:
    sentimento='Positivo'
elif qtd_negativa > 3*qtd_positiva:
    sentimento='Negativo'

print(f'O sentimento sobre o filme foi: {sentimento}')
```

Cujo resultado da execução é: O sentimento sobre o filme foi: Neutro.

Considerações finais

O que você aprendeu neste conteúdo?

- Tokenização, marcação de parte do discurso e reconhecimento de entidade nomeada.
- Técnicas para PLN: CBOW e Skip-Gram.
- Aplicação em Python usando a biblioteca regex.

Explore +

Confira as indicações que separamos especialmente para você!

- Acesse o site oficial da Microsoft e pesquise por **Regular Expression Language - Quick Reference**. Nele você vai aprofundar os seus conhecimentos sobre os recursos de Regex e encontrará mais exemplos que vão ajudar você a entender esse assunto melhor.
- Acesse o site oficial da Oracle e procure por **O que é Processamento de Linguagem Natural?** Lá, você vai encontrar diversos exemplos de aplicações usadas no mercado e, certamente, vai ampliar a sua visão sobre esse assunto tão interessante e atual.

Referências

BIRD, S.; KLEIN, E.; LOPER, E. **Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit**. O'Reilly Media, 2009.

JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition**. Pearson Education, 2020.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge University Press, 2008.

PYTHON. **Regular expression operations**. Consultado na internet em: 23 out. 2023.